```cpp
/**   多项式（Poly 手写取模）
 *    2021-06-16: https://codeforces.com/gym/103119/submission/119653035
**/
constexpr int C = 1024;
constexpr int P = 998244353;
std::vector<int> rev, roots{0, 1};
int power(int a, int b) {
    int res = 1;
    for (; b; b >>= 1, a = 1ll * a * a % P)
        if (b & 1)
            res = 1ll * res * a % P;
    return res;
}
void dft(std::vector<int> &a) {
    int n = a.size();
    if (int(rev.size()) != n) {
        int k = __builtin_ctz(n) - 1;
        rev.resize(n);
        for (int i = 0; i < n; ++i)
            rev[i] = rev[i >> 1] >> 1 | (i & 1) << k;
    }
    for (int i = 0; i < n; ++i)
        if (rev[i] < i)
            std::swap(a[i], a[rev[i]]);
    if (int(roots.size()) < n) {
        int k = __builtin_ctz(roots.size());
        roots.resize(n);
        while ((1 << k) < n) {
            int e = power(3, (P - 1) >> (k + 1));
            for (int i = 1 << (k - 1); i < (1 << k); ++i) {
                roots[2 * i] = roots[i];
                roots[2 * i + 1] = 1ll * roots[i] * e % P;
            }
            ++k;
        }
    }
    for (int k = 1; k < n; k *= 2) {
        for (int i = 0; i < n; i += 2 * k) {
            for (int j = 0; j < k; ++j) {
                int u = a[i + j];
                int v = 1ll * a[i + j + k] * roots[k + j] % P;
                int x = u + v;
                if (x >= P)
                    x -= P;
                a[i + j] = x;
                x = u - v;
                if (x < 0)
                    x += P;
                a[i + j + k] = x;
            }
        }
```

```
    }
}
void idft(std::vector<int> &a) {
    int n = a.size();
    std::reverse(a.begin() + 1, a.end());
    dft(a);
    int inv = power(n, P - 2);
    for (int i = 0; i < n; ++i)
        a[i] = 1ll * a[i] * inv % P;
}
struct Poly {
    std::vector<int> a;
    Poly() {}
    Poly(int a0) {
        if (a0)
            a = {a0};
    }
    Poly(const std::vector<int> &a1) : a(a1) {
        while (!a.empty() && !a.back())
            a.pop_back();
    }
    int size() const {
        return a.size();
    }
    int operator[](int idx) const {
        if (idx < 0 || idx >= size())
            return 0;
        return a[idx];
    }
    Poly mulxk(int k) const {
        auto b = a;
        b.insert(b.begin(), k, 0);
        return Poly(b);
    }
    Poly modxk(int k) const {
        k = std::min(k, size());
        return Poly(std::vector<int>(a.begin(), a.begin() + k));
    }
    Poly divxk(int k) const {
        if (size() <= k)
            return Poly();
        return Poly(std::vector<int>(a.begin() + k, a.end()));
    }
    friend Poly operator+(const Poly a, const Poly &b) {
        std::vector<int> res(std::max(a.size(), b.size()));
        for (int i = 0; i < int(res.size()); ++i) {
            res[i] = a[i] + b[i];
            if (res[i] >= P)
                res[i] -= P;
        }
        return Poly(res);
    }
```

```
    friend Poly operator-(const Poly a, const Poly &b) {
        std::vector<int> res(std::max(a.size(), b.size()));
        for (int i = 0; i < int(res.size()); ++i) {
            res[i] = a[i] - b[i];
            if (res[i] < 0)
                res[i] += P;
        }
        return Poly(res);
    }
    friend Poly operator*(Poly a, Poly b) {
        int sz = 1, tot = a.size() + b.size() - 1;
        while (sz < tot)
            sz *= 2;
        a.a.resize(sz);
        b.a.resize(sz);
        dft(a.a);
        dft(b.a);
        for (int i = 0; i < sz; ++i)
            a.a[i] = 1ll * a[i] * b[i] % P;
        idft(a.a);
        return Poly(a.a);
    }
    Poly &operator+=(Poly b) {
        return (*this) = (*this) + b;
    }
    Poly &operator-=(Poly b) {
        return (*this) = (*this) - b;
    }
    Poly &operator*=(Poly b) {
        return (*this) = (*this) * b;
    }
    Poly deriv() const {
        if (a.empty())
            return Poly();
        std::vector<int> res(size() - 1);
        for (int i = 0; i < size() - 1; ++i)
            res[i] = 1ll * (i + 1) * a[i + 1] % P;
        return Poly(res);
    }
    Poly integr() const {
        if (a.empty())
            return Poly();
        std::vector<int> res(size() + 1);
        for (int i = 0; i < size(); ++i)
            res[i + 1] = 1ll * a[i] * power(i + 1, P - 2) % P;
        return Poly(res);
    }
    Poly inv(int m) const {
        Poly x(power(a[0], P - 2));
        int k = 1;
        while (k < m) {
            k *= 2;
```

```
            x = (x * (2 - modxk(k) * x)).modxk(k);
        }
        return x.modxk(m);
    }
    Poly log(int m) const {
        return (deriv() * inv(m)).integr().modxk(m);
    }
    Poly exp(int m) const {
        Poly x(1);
        int k = 1;
        while (k < m) {
            k *= 2;
            x = (x * (1 - x.log(k) + modxk(k))).modxk(k);
        }
        return x.modxk(m);
    }
    Poly sqrt(int m) const {
        Poly x(1);
        int k = 1;
        while (k < m) {
            k *= 2;
            x = (x + (modxk(k) * x.inv(k)).modxk(k)) * ((P + 1) / 2);
        }
        return x.modxk(m);
    }
    Poly mulT(Poly b) const {
        if (b.size() == 0)
            return Poly();
        int n = b.size();
        std::reverse(b.a.begin(), b.a.end());
        return ((*this) * b).divxk(n - 1);
    }
    std::vector<int> eval(std::vector<int> x) const {
        if (size() == 0)
            return std::vector<int>(x.size(), 0);
        const int n = std::max(int(x.size()), size());
        std::vector<Poly> q(4 * n);
        std::vector<int> ans(x.size());
        x.resize(n);
        std::function<void(int, int, int)> build = [&](int p, int l, int r) {
            if (r - l == 1) {
                q[p] = std::vector<int>{1, (P - x[l]) % P};
            } else {
                int m = (l + r) / 2;
                build(2 * p, l, m);
                build(2 * p + 1, m, r);
                q[p] = q[2 * p] * q[2 * p + 1];
            }
        };
        build(1, 0, n);
        std::function<void(int, int, int, const Poly &)> work = [&](int p, int l,
int r, const Poly &num) {
```

```
            if (r - l == 1) {
                if (l < int(ans.size()))
                    ans[l] = num[0];
            } else {
                int m = (l + r) / 2;
                work(2 * p, l, m, num.mulT(q[2 * p + 1]).modxk(m - l));
                work(2 * p + 1, m, r, num.mulT(q[2 * p]).modxk(r - m));
            }
        };
        work(1, 0, n, mulT(q[1].inv(n)));
        return ans;
    }
};
using i64 = long long;
void dft(std::vector<std::vector<int>> &a) {
    int n = a.size();
    for (auto &v : a) {
        dft(v);
    }
    for (int i = 0; i < int(a[0].size()); i++) {
        std::vector<int> v(n);
        for (int j = 0; j < n; j++) {
            v[j] = a[j][i];
        }
        dft(v);
        for (int j = 0; j < n; j++) {
            a[j][i] = v[j];
        }
    }
}
void idft(std::vector<std::vector<int>> &a) {
    int n = a.size();
    for (auto &v : a) {
        idft(v);
    }
    for (int i = 0; i < int(a[0].size()); i++) {
        std::vector<int> v(n);
        for (int j = 0; j < n; j++) {
            v[j] = a[j][i];
        }
        idft(v);
        for (int j = 0; j < n; j++) {
            a[j][i] = v[j];
        }
    }
}
auto inv(const std::vector<std::vector<int>> &a) {
    int m = 1;
    std::vector g(1, std::vector{Poly(a[0]).inv(C).a});
    while (m < C) {
        std::vector a0(4 * m, std::vector<int>(4 * C));
        for (int i = 0; i < 2 * m; i++) {
```

```
            for (int j = 0; j < C; j++) {
                a0[i][j] = a[i][j];
            }
        }
        dft(a0);
        g.resize(4 * m);
        for (auto &v : g) {
            v.resize(4 * C);
        }
        dft(g);
        for (int i = 0; i < 4 * m; i++) {
            for (int j = 0; j < 4 * C; j++) {
                g[i][j] = i64(g[i][j]) * (2 + i64(P - a0[i][j]) * g[i][j] % P) % P;
            }
        }
        idft(g);
        m *= 2;
        g.resize(m);
        for (auto &v : g) {
            v.resize(C);
        }
    }
    return g;
}
```